Distributed Computing (CE301)

Lab ESE Master Programs: 10 codes that cover all 340 chits

Every chit in the question bank is one of 10 topics. The application domain (TrustMesh, Donation Management, Online Banking, and so on) never changes the code. Only three things change from chit to chit: **the CSV file name**, **the column names** and **small extra requirements**. Learn these 10 programs, and you can answer any chit.

Each program in this document has: the chit wording it answers, the CSV format, a sample CSV, the complete code (tested), the real output, how it works, and exactly what to change if the question changes.

Which program answers which chit

#	Topic in the chit	Chits	CSV pattern	Program file
1	RMI / RPC	36	`rpc_calls_N.csv`	`rpc.py`
2	Bully algorithm	36	`bully_cluster_N.csv`	`bully.py`
3	Ring algorithm	34	`ring_nodes_N.csv`	`ring.py`
4	Cristian's algorithm	35	`cristian_clocks_N.csv`	`cristian.py`
5	Berkeley's algorithm	35	`node_clocks_N.csv`	`berkeley.py`
6	Lamport's logical clock	32	`events_N.csv`	`lamport.py`
7	Vector clocks	32	`vector_events_N.csv`	`vector_clock.py`
8	Multithreading + mutex	34	`tasks_N.csv`	`multithread.py`
9	Load balancing	33	`workload_N.csv`	`load_balancer.py`
10	MPI matrix multiplication	33	`matrix_N.csv`	`mpi_matrix.py`
	Total	**340**		

Quick keyword finder (what to look for in the chit)

If the chit says...	Use
remote method, client-server, dispatcher, marshals	1. RPC
ELECTION, OK, COORDINATOR, bullies lower-ID nodes	2. Bully
token, ring nodes, next_node_id	3. Ring
RTT, T_server, roundtrip, adjusted client time	4. Cristian
discard faulty outliers, average time, adjustment per node	5. Berkeley
L = max(L, T_msg) + 1, total causal order	6. Lamport
[V1, V2, V3], causally related or concurrent	7. Vector clocks
thread locks, mutex, race conditions	8. Multithreading
Round-Robin, Least-Connections, workers, jobs	9. Load balancing
scatter-gather, matrix, row dot products	10. MPI matrix

Setting up and running in VS Code

Do this once at the start of the lab. It takes about two minutes.

1. **Make a folder** for the lab, for example **DC_Lab** on the Desktop.
2. In VS Code choose **File > Open Folder...** and open that folder. Open the **folder**, not just a single file. The terminal then starts inside the folder, and Python can find your CSV files.
3. Create the CSV first. Click the **New File** icon in the Explorer panel and type the **exact file name from the chit** (for example `rpc_calls_1.csv`). The first line is the header with the column names, then one row per line. Save with **Ctrl+S**.
4. Create the program file in the same folder (for example `rpc.py`). Type or paste the code and save.
5. Open the terminal with **Ctrl + `** (the key below Esc) or **Terminal > New Terminal**.
6. Run the program: `python rpc.py` on Windows, `python3 rpc.py` on Linux or macOS. If `python` is not recognised on Windows, try `py rpc.py`. The triangle button at the top right of the editor also runs the file.
7. Bully and Ring ask for an initiator node. Type the node ID in the terminal and press **Enter**.
8. Keep the output visible in the terminal for the examiner. Drag the terminal panel taller if the output is long.

Rules that prevent silly mistakes

- The CSV **must be in the same folder** as the program, and the name in `CSV_FILE` must match it exactly.
- Column names in the CSV must match the names in the code, including capitals and underscores.
- Do not name your own file `csv.py`, `threading.py`, `time.py` or `json.py`. Python would import your file instead of the library.
- If you use Excel to make the CSV, save as **CSV (Comma delimited)**. It is easier to type it in VS Code.
- Windows can hide extensions and save `data.csv.txt`. Turn on **File name extensions** in File Explorer to check.
- Spaces after commas (as in the chit's sample row) are fine. The programs strip them.
- Use 4 spaces for indentation. Do not mix tabs and spaces.

Common errors and the fix

Error message	Cause	Fix
`FileNotFoundError`	Wrong folder or wrong file name	Open the folder in VS Code. Compare the name with `CSV_FILE`.
`KeyError: 'node_id'`	Header in the CSV differs from the code	Make the header identical to the chit's column list.
`ValueError: invalid literal for int()`	Blank line, repeated header or text in a number column	Delete blank lines. Check the data rows.
`IndentationError`	Mixed tabs and spaces	Select all and re-indent with 4 spaces.
`'python' is not recognized`	Python not on PATH	Use `py file.py` or `python3 file.py`.
`EOFError`	input() has no terminal to read from	Run in the VS Code terminal, not in an output pane.

The 6-step routine for every chit

1. Match the topic with the keyword table and pick the program.
2. Write down the **CSV file name** and the **column names** from the chit.
3. Create the CSV with the header and 5 to 8 rows. Include an awkward row: a dead node, a faulty clock, an unknown method.
4. Type the program. Change `CSV_FILE` and any column names that differ.
5. Add one line at the top: `print("Application domain: <name from the chit>")`.
6. Run it. Compare the output with the chit's *Expected Output / Verification* line. Every label it mentions must appear in your output.

1. RMI / RPC - Remote Procedure Call

Program file	`rpc.py` (36 chits of this type)
CSV file	`rpc_calls_1.csv` (the number changes in each chit)
Columns	`call_id, client_name, method_name, param_value, timestamp`
Chit wording	Simulate an RPC/RMI client-server dispatcher; read method calls from CSV; marshal arguments; execute the remote function; print the returned result.
Output must show	[DISPATCH] -> [MARSHALLING] -> [SERVER EXECUTION] -> [RETURN]

How it works

- Each CSV row is one remote call made by a client.
- **Marshalling** = packing the method name and argument into one message (a JSON string). The server **unmarshals** it and runs the function.
- The **dispatcher** is a dictionary `methods` that maps the method name (text from the CSV) to a real Python function. A name that is not in the dictionary returns `"Method not found"`.
- The result goes back to the client, which prints it.

Sample CSV (rpc_calls_1.csv)

```
call_id, client_name, method_name, param_value, timestamp
101, Client_A, processData, 450, 10:00:01
102, Client_B, calculateSquare, 12, 10:00:03
103, Client_A, calculateCube, 5, 10:00:05
104, Client_C, addData, 50, 10:00:08
105, Client_B, deleteData, 7, 10:00:10
```

Program (rpc.py)

```
import csv
import json

CSV_FILE = "rpc_calls_1.csv"          # <-- change the filename here

# ---------- SERVER SIDE: remote procedures ----------
def processData(x):
    return x * 2

def calculateSquare(x):
    return x * x

def calculateCube(x):
    return x ** 3

def addData(x):
    return x + 100

# Dispatcher table: method name (text from the CSV) -> real function
methods = {
    "processData": processData,
    "calculateSquare": calculateSquare,
    "calculateCube": calculateCube,
    "addData": addData,
}

def server_dispatch(message):
    request = json.loads(message)                    # unmarshal
    method = request["method"]
    value = request["param"]
    print("[SERVER EXECUTION] Executing", method, "with parameter", value)
    if method in methods:
        return methods[method](value)
    return "Method not found"

# ---------- CLIENT SIDE: read the calls from the CSV ----------
with open(CSV_FILE, "r") as f:
    reader = csv.DictReader(f, skipinitialspace=True)
    for row in reader:
        call_id = row["call_id"].strip()
        client = row["client_name"].strip()
        method = row["method_name"].strip()
```

```
        value = int(row["param_value"])
        stamp = row["timestamp"].strip()

        print("\n[DISPATCH] Call", call_id, "at", stamp)
        print("Client", client, "invoking method:", method)

        message = json.dumps({"method": method, "param": value})   # marshal
        print("[MARSHALLING] Packed request ->", message)

        result = server_dispatch(message)

        print("[RETURN] Result sent back to", client, "->", result)
```

Sample output

```
[DISPATCH] Call 101 at 10:00:01
Client Client_A invoking method: processData
[MARSHALLING] Packed request -> {"method": "processData", "param": 450}
[SERVER EXECUTION] Executing processData with parameter 450
[RETURN] Result sent back to Client_A -> 900

[DISPATCH] Call 102 at 10:00:03
Client Client_B invoking method: calculateSquare
[MARSHALLING] Packed request -> {"method": "calculateSquare", "param": 12}
[SERVER EXECUTION] Executing calculateSquare with parameter 12
[RETURN] Result sent back to Client_B -> 144

[DISPATCH] Call 103 at 10:00:05
Client Client_A invoking method: calculateCube
[MARSHALLING] Packed request -> {"method": "calculateCube", "param": 5}
[SERVER EXECUTION] Executing calculateCube with parameter 5
[RETURN] Result sent back to Client_A -> 125

[DISPATCH] Call 104 at 10:00:08
Client Client_C invoking method: addData
[MARSHALLING] Packed request -> {"method": "addData", "param": 50}
[SERVER EXECUTION] Executing addData with parameter 50
[RETURN] Result sent back to Client_C -> 150

[DISPATCH] Call 105 at 10:00:10
Client Client_B invoking method: deleteData
[MARSHALLING] Packed request -> {"method": "deleteData", "param": 7}
[SERVER EXECUTION] Executing deleteData with parameter 7
[RETURN] Result sent back to Client_B -> Method not found
```

If the question changes

If the question says...	Change this
Chit uses other method names	Write the function and add one line to the `methods` dictionary.
param_value is not an integer	Change `int(row["param_value"])` to `float(...)` or plain text.
Two or more parameters (param_a, param_b)	Read both columns, send `"params": [a, b]` in the JSON and call `methods[m](*params)`.
Different operation	Only edit the body of the server function.
Extra column in the CSV	Read it with `row["col"]` and print it in the DISPATCH block.

Watch out

- Keep the labels exactly as the chit prints them: [DISPATCH], [SERVER EXECUTION], [RETURN].
- The chit does not say what `processData` does. Here it doubles the value. Tell the examiner your assumption.
- Keep one row with an unknown method (like deleteData) to show the error handling.

2. Bully Algorithm - Leader Election

Program file	`bully.py` (36 chits of this type)
CSV file	`bully_cluster_2.csv` (the number changes in each chit)
Columns	`node_id, is_alive, is_current_coordinator`
Chit wording	A node detects coordinator failure and bullies lower-ID nodes to become leader. Print ELECTION to higher nodes, OK replies, and COORDINATOR broadcast by the highest alive node.
Output must show	ELECTION -> OK -> COORDINATOR

How it works

- The initiator sends **ELECTION** only to nodes with a **higher** ID.
- An alive higher node replies **OK** and takes over (it starts its own election). A dead node gives no reply.
- A node that receives no OK from anybody wins. It sends **COORDINATOR** to every other alive node.
- The function `election(starter)` calls itself for every node that answered OK. That is the takeover.

Sample CSV (bully_cluster_2.csv)

```
node_id,is_alive,is_current_coordinator
1,TRUE,FALSE
2,TRUE,FALSE
3,TRUE,FALSE
4,TRUE,FALSE
5,TRUE,FALSE
6,FALSE,TRUE
```

Program (bully.py)

```
import csv

CSV_FILE = "bully_cluster_2.csv"      # <-- change the filename here

nodes = {}
with open(CSV_FILE, "r") as f:
    for row in csv.DictReader(f, skipinitialspace=True):
        nid = int(row["node_id"])
        nodes[nid] = {
            "alive": row["is_alive"].strip().upper() == "TRUE",
            "coord": row["is_current_coordinator"].strip().upper() == "TRUE",
        }

print("Node  Alive  Coordinator")
for nid in sorted(nodes):
    print(f"{nid:<5} {str(nodes[nid]['alive']):<6} {nodes[nid]['coord']}")

old = next((n for n in nodes if nodes[n]["coord"]), None)
if old is not None and not nodes[old]["alive"]:
    print("\nCoordinator failure detected: Node", old, "is DOWN")

initiator = int(input("\nEnter election initiator node ID: "))
if initiator not in nodes or not nodes[initiator]["alive"]:
    print("Initiator must be an alive node!")
    raise SystemExit

started = set()

def election(starter):
    started.add(starter)
    print(f"\nNode {starter} starts ELECTION")
    responders = []
    for h in sorted(n for n in nodes if n > starter):     # only HIGHER ids
        print(f"  Node {starter} -> ELECTION -> Node {h}")
        if nodes[h]["alive"]:
            print(f"  Node {h} -> OK -> Node {starter}")
            responders.append(h)
        else:
            print(f"  Node {h} is DOWN (no reply)")
    if not responders:
        print(f"Node {starter} received no OK, so it WINS")
        return starter
    winner = starter
    for r in responders:
```

```
        if r not in started:
            print(f"Node {starter} drops out; Node {r} takes over the election")
            winner = max(winner, election(r))
    return winner

winner = election(initiator)

print(f"\nNode {winner} is the new COORDINATOR")
for n in sorted(nodes):
    if n != winner and nodes[n]["alive"]:
        print(f"Node {winner} -> COORDINATOR -> Node {n}")

for n in nodes:
    nodes[n]["coord"] = (n == winner)
print("Final coordinator:", winner)
```

Sample output (Typed at the prompt: 3 (node 3 starts the election))

```
Node  Alive  Coordinator
1     True   False
2     True   False
3     True   False
4     True   False
5     True   False
6     False  True

Coordinator failure detected: Node 6 is DOWN

Enter election initiator node ID:
Node 3 starts ELECTION
  Node 3 -> ELECTION -> Node 4
  Node 4 -> OK -> Node 3
  Node 3 -> ELECTION -> Node 5
  Node 5 -> OK -> Node 3
  Node 3 -> ELECTION -> Node 6
  Node 6 is DOWN (no reply)
Node 3 drops out; Node 4 takes over the election

Node 4 starts ELECTION
  Node 4 -> ELECTION -> Node 5
  Node 5 -> OK -> Node 4
  Node 4 -> ELECTION -> Node 6
  Node 6 is DOWN (no reply)
Node 4 drops out; Node 5 takes over the election

Node 5 starts ELECTION
  Node 5 -> ELECTION -> Node 6
  Node 6 is DOWN (no reply)
Node 5 received no OK, so it WINS

Node 5 is the new COORDINATOR
Node 5 -> COORDINATOR -> Node 1
Node 5 -> COORDINATOR -> Node 2
Node 5 -> COORDINATOR -> Node 3
Node 5 -> COORDINATOR -> Node 4
Final coordinator: 5
```

If the question changes

If the question says...	Change this
A different initiator	Type another ID at the prompt, or hard-code `initiator = 2`.
Initiator must be chosen automatically	`initiator = min(n for n in nodes if nodes[n]["alive"])`
Highest priority instead of highest ID	Read the priority column into `nodes[nid]["prio"]` and change the test to `nodes[n]["prio"] > nodes[starter]["prio"]`.
Status written as UP/DOWN or 1/0	Change the `== "TRUE"` test.
More nodes	No change. Just add rows.

Watch out

- The initiator must be alive. The program stops with a message if it is not.
- The old coordinator is shown as failed only if the CSV marks it `is_alive = FALSE`.
- Print the node table first. It shows the examiner you read the CSV correctly.

3. Ring Algorithm - Leader Election

Program file	`ring.py` (34 chits of this type)
CSV file	`ring_nodes_15.csv` (the number changes in each chit)
Columns	`node_id, next_node_id, is_active, priority_rank`
Chit wording	Simulate coordinator failure and circulate election tokens among active ring nodes. Print the failure, token circulation, and the election of the node with the highest ID.
Output must show	Detected failure -> ELECTION token circulation -> COORDINATOR (highest ID)

How it works

- Each CSV row gives the **next** node in the ring. Inactive nodes are skipped while passing the token.
- The initiator sends an **ELECTION** message holding its own ID. Every active node adds its ID to the list and forwards it.
- When the token comes back to the initiator, the **highest ID** in the list is the winner.
- A **COORDINATOR** message then goes once round the ring so everyone learns the winner.

Sample CSV (ring_nodes_15.csv)

```
node_id,next_node_id,is_active,priority_rank
1,2,TRUE,85
2,3,TRUE,70
3,4,TRUE,90
4,5,TRUE,60
5,1,FALSE,95
```

Program (ring.py)

```
import csv

CSV_FILE = "ring_nodes_15.csv"        # <-- change the filename here

nodes = {}
with open(CSV_FILE, "r") as f:
    for row in csv.DictReader(f, skipinitialspace=True):
        nid = int(row["node_id"])
        nodes[nid] = {
            "next": int(row["next_node_id"]),
            "active": row["is_active"].strip().upper() == "TRUE",
            "priority": int(row["priority_rank"]),
        }

print("Node  Next  Active  Priority")
for nid in sorted(nodes):
    d = nodes[nid]
    print(f"{nid:<5} {d['next']:<5} {str(d['active']):<7} {d['priority']}")

old = max(nodes)                      # old coordinator = highest id
if not nodes[old]["active"]:
    print("\nCoordinator failure detected: Node", old, "is DOWN")

initiator = int(input("\nEnter election initiator node ID: "))
if initiator not in nodes or not nodes[initiator]["active"]:
    print("Initiator must be an active node!")
    raise SystemExit

# ---------- ELECTION token goes round the ring ----------
token = [initiator]
current = initiator
while True:
    nxt = nodes[current]["next"]
    while not nodes[nxt]["active"]:
        print(f"Node {current}: Node {nxt} is DOWN, skipping")
        nxt = nodes[nxt]["next"]
    if nxt == initiator:
        print(f"Node {current} -> ELECTION {token} -> Node {nxt} (token back at initiator)")
        break
    print(f"Node {current} -> ELECTION {token} -> Node {nxt}")
    token.append(nxt)
    current = nxt
    if len(token) > len(nodes):
        print("Ring is broken!")
```

```
        raise SystemExit

winner = max(token)                    # highest ID wins
# winner = max(token, key=lambda n: nodes[n]["priority"])   # <- if PRIORITY is asked
print("\nIDs collected in token:", token)
print("Elected coordinator: Node", winner)

# ---------- COORDINATOR message goes round the ring ----------
current = winner
while True:
    nxt = nodes[current]["next"]
    while not nodes[nxt]["active"]:
        nxt = nodes[nxt]["next"]
    if nxt == winner:
        break
    print(f"Node {current} -> COORDINATOR({winner}) -> Node {nxt}")
    current = nxt
print("Election complete. New coordinator:", winner)
```

Sample output (Typed at the prompt: 2 (node 2 starts the election))

```
Node  Next  Active  Priority
1     2     True    85
2     3     True    70
3     4     True    90
4     5     True    60
5     1     False   95

Coordinator failure detected: Node 5 is DOWN

Enter election initiator node ID: Node 2 -> ELECTION [2] -> Node 3
Node 3 -> ELECTION [2, 3] -> Node 4
Node 4: Node 5 is DOWN, skipping
Node 4 -> ELECTION [2, 3, 4] -> Node 1
Node 1 -> ELECTION [2, 3, 4, 1] -> Node 2 (token back at initiator)

IDs collected in token: [2, 3, 4, 1]
Elected coordinator: Node 4
Node 4 -> COORDINATOR(4) -> Node 1
Node 1 -> COORDINATOR(4) -> Node 2
Node 2 -> COORDINATOR(4) -> Node 3
Election complete. New coordinator: 4
```

If the question changes

If the question says...	Change this
Winner by priority_rank	Use `max(token, key=lambda n: nodes[n]["priority"])` (the commented line in the program).
Different starting node	Type a different ID at the prompt.
CSV has no next_node_id	Build the ring from sorted IDs: `next = the following ID, wrapping to the first`.
Different active/inactive wording	Change the `== "TRUE"` test.

Watch out

- The chit's expected output says **highest ID**. The priority_rank column is only used if the question says so.
- The program assumes the links form a closed ring. A broken ring is caught with a message.
- The failed coordinator is taken as the highest-ID node. If your chit says which node failed, print that.

4. Cristian's Algorithm - Clock Synchronization

Program file	`cristian.py` (35 chits of this type)
CSV file	`cristian_clocks_8.csv` (the number changes in each chit)
Columns	`client_id, t0_send_ms, t_server_ms, t1_recv_ms`
Chit wording	Calculate round-trip time (RTT), time server drift and adjusted client time from CSV records.
Output must show	RTT = T1 - T0, estimated server time = Tserver + RTT/2, clock offset

Formulas / rules

- `RTT = T1 - T0`
- `T_estimated = T_server + RTT / 2`
- `Offset = T_estimated - T1`
- `Adjusted client time = T1 + Offset`

How it works

- T0 is when the client sent the request, T_server is the server's clock reading, T1 is when the reply arrived.
- The server's reading is assumed to be taken halfway, so the client adds **RTT / 2** to it.
- Offset > 0 means the client clock is **behind**. Offset < 0 means it is **ahead**.
- Every CSV row is a different client, so the loop handles any number of clients.

Sample CSV (cristian_clocks_8.csv)

```
client_id,t0_send_ms,t_server_ms,t1_recv_ms
C1,1000,1050,1100
C2,2000,2080,2120
C3,3000,3010,3050
```

Program (cristian.py)

```
import csv

CSV_FILE = "cristian_clocks_8.csv"    # <-- change the filename here

with open(CSV_FILE, "r") as f:
    for row in csv.DictReader(f, skipinitialspace=True):
        client = row["client_id"].strip()
        t0 = float(row["t0_send_ms"])      # client send time
        ts = float(row["t_server_ms"])     # time returned by server
        t1 = float(row["t1_recv_ms"])      # client receive time

        rtt = t1 - t0
        estimated = ts + rtt / 2
        offset = estimated - t1            # + : client is behind, - : client is ahead
        adjusted = t1 + offset

        print("\nClient:", client)
        print(f"RTT = T1 - T0 = {t1} - {t0} = {rtt} ms")
        print(f"Estimated server time = Tserver + RTT/2 = {ts} + {rtt/2} = {estimated} ms")
        print(f"Clock offset (drift) = {estimated} - {t1} = {offset} ms")
        print(f"Adjusted client time = {t1} + ({offset}) = {adjusted} ms")
```

Sample output

```
Client: C1
RTT = T1 - T0 = 1100.0 - 1000.0 = 100.0 ms
Estimated server time = Tserver + RTT/2 = 1050.0 + 50.0 = 1100.0 ms
Clock offset (drift) = 1100.0 - 1100.0 = 0.0 ms
Adjusted client time = 1100.0 + (0.0) = 1100.0 ms

Client: C2
RTT = T1 - T0 = 2120.0 - 2000.0 = 120.0 ms
Estimated server time = Tserver + RTT/2 = 2080.0 + 60.0 = 2140.0 ms
Clock offset (drift) = 2140.0 - 2120.0 = 20.0 ms
Adjusted client time = 2120.0 + (20.0) = 2140.0 ms

Client: C3
RTT = T1 - T0 = 3050.0 - 3000.0 = 50.0 ms
Estimated server time = Tserver + RTT/2 = 3010.0 + 25.0 = 3035.0 ms
Clock offset (drift) = 3035.0 - 3050.0 = -15.0 ms
```

```
Adjusted client time = 3050.0 + (-15.0) = 3035.0 ms
```

If the question changes

If the question says...	Change this
Seconds instead of milliseconds	Formula is the same. Change the units in the print text only.
Average offset of all clients	Add `total += offset` in the loop, then print `total / count` after it.
Reject slow replies	`if rtt > LIMIT: print('discarded'); continue`
Time given as HH:MM:SS	Convert to seconds first, calculate, then convert back.

Watch out

- Print the formula with the numbers substituted. The program already does it. Markers love it.
- Keep the same unit everywhere (all ms or all s).

5. Berkeley's Algorithm - Physical Clock Synchronization

Program file	`berkeley.py` (35 chits of this type)
CSV file	`node_clocks_3.csv` (the number changes in each chit)
Columns	`node_id, clock_time_sec, is_faulty`
Chit wording	Read node times from CSV, discard faulty outliers, compute the average time and the relative adjustment per node.
Output must show	Polled times -> discarded outliers -> average -> adjustment per node

Formulas / rules

- `Average = sum of valid clocks / number of valid clocks`
- `Adjustment = Average - NodeClock`

How it works

- The master polls every node and prints its time.
- Nodes flagged `is_faulty = TRUE` are discarded. With `OUTLIER_LIMIT` set, clocks too far from the median are discarded too.
- The average of the remaining clocks is the new common time.
- Each node gets an adjustment (+ means move forward, - means move back). Excluded nodes are marked.

Sample CSV (node_clocks_3.csv)

```
node_id,clock_time_sec,is_faulty
N1,3600,FALSE
N2,3610,FALSE
N3,3590,FALSE
N4,9000,TRUE
N5,3620,FALSE
```

Program (berkeley.py)

```
import csv
import statistics

CSV_FILE = "node_clocks_3.csv"        # <-- change the filename here
OUTLIER_LIMIT = None                  # e.g. 300 -> ALSO discard clocks more than 300 s from the median

def hms(sec):
    sec = int(round(sec))
    return f"{sec // 3600:02d}:{(sec % 3600) // 60:02d}:{sec % 60:02d}"

nodes = []
with open(CSV_FILE, "r") as f:
    for row in csv.DictReader(f, skipinitialspace=True):
        nodes.append({
            "id": row["node_id"].strip(),
            "time": float(row["clock_time_sec"]),
            "faulty": row["is_faulty"].strip().upper() == "TRUE",
        })

print("Master polls all nodes:")
for n in nodes:
    print(f"  {n['id']}: {n['time']} s ({hms(n['time'])})")

# optional statistical outlier check
good = [n["time"] for n in nodes if not n["faulty"]]
if OUTLIER_LIMIT is not None and good:
    med = statistics.median(good)
    for n in nodes:
        if abs(n["time"] - med) > OUTLIER_LIMIT:
            n["faulty"] = True

valid = []
print("\nFiltering:")
for n in nodes:
    if n["faulty"]:
        print(f"  {n['id']}: DISCARDED as faulty/outlier ({n['time']} s)")
    else:
        valid.append(n["time"])
```

```
        print(f"  {n['id']}: accepted")

if not valid:
    print("No valid clocks available!")
    raise SystemExit

average = sum(valid) / len(valid)
print(f"\nAverage = sum / count = {sum(valid)} / {len(valid)} = {average} s ({hms(average)})")

print("\nAdjustment = Average - NodeClock")
for n in nodes:
    adj = average - n["time"]
    note = "  (excluded from average)" if n["faulty"] else ""
    print(f"  {n['id']}: {average} - {n['time']} = {adj:+} s -> new time {n['time'] + adj} s{note}")
```

Sample output

```
Master polls all nodes:
  N1: 3600.0 s (01:00:00)
  N2: 3610.0 s (01:00:10)
  N3: 3590.0 s (00:59:50)
  N4: 9000.0 s (02:30:00)
  N5: 3620.0 s (01:00:20)

Filtering:
  N1: accepted
  N2: accepted
  N3: accepted
  N4: DISCARDED as faulty/outlier (9000.0 s)
  N5: accepted

Average = sum / count = 14420.0 / 4 = 3605.0 s (01:00:05)

Adjustment = Average - NodeClock
  N1: 3605.0 - 3600.0 = +5.0 s -> new time 3605.0 s
  N2: 3605.0 - 3610.0 = -5.0 s -> new time 3605.0 s
  N3: 3605.0 - 3590.0 = +15.0 s -> new time 3605.0 s
  N4: 3605.0 - 9000.0 = -5395.0 s -> new time 3605.0 s  (excluded from average)
  N5: 3605.0 - 3620.0 = -15.0 s -> new time 3605.0 s
```

If the question changes

If the question says...	Change this
No is_faulty column; outliers by value	Set `OUTLIER_LIMIT = 300` (any limit in seconds) and set every flag to FALSE.
Times as HH:MM:SS	Convert to seconds with `h*3600 + m*60 + s`. The `hms()` helper converts back.
Master has its own clock row	No change. It is just another node.
Different column names	Change the three `row[...]` lookups.

Watch out

- Say clearly which node(s) you discarded and why.
- All nodes, including discarded ones, get an adjustment printed. Discarded ones are labelled.

6. Lamport's Logical Clock

Program file	`lamport.py` (32 chits of this type)
CSV file	`events_7.csv` (the number changes in each chit)
Columns	`event_id, process_id, event_type, message_to, initial_clock`
Chit wording	Assign Lamport timestamps to a sequence of events from CSV and print their total causal order.
Output must show	Step-by-step L updates using L = max(L, T_msg) + 1, then the final sorted order

Formulas / rules

- Internal event: L = L + 1
- Send event: L = L + 1 (message carries L)
- Receive event: L = max(L, T_msg) + 1

How it works

- Each process has its own clock. It starts at `initial_clock` the first time the process appears.
- **SEND**: add 1, and put that timestamp into the message queue for the receiver.
- **RECEIVE**: take the oldest waiting message from that sender, then `max(own, message) + 1`.
- At the end all events are sorted by timestamp. Equal timestamps are ordered by process ID.

Sample CSV (events_7.csv)

```
event_id,process_id,event_type,message_to,initial_clock
E1,P1,SEND,P2,0
E2,P2,INTERNAL,-,0
E3,P2,RECEIVE,P1,0
E4,P2,SEND,P3,0
E5,P3,RECEIVE,P2,0
E6,P1,INTERNAL,-,0
E7,P3,SEND,P1,0
E8,P1,RECEIVE,P3,0
```

Program (lamport.py)

```
import csv

CSV_FILE = "events_7.csv"            # <-- change the filename here

clocks = {}        # process -> current Lamport clock
queues = {}        # (sender, receiver) -> timestamps of messages in transit
log = []           # (timestamp, process, event)

print("Step-by-step clock updates")
with open(CSV_FILE, "r") as f:
    for row in csv.DictReader(f, skipinitialspace=True):
        eid = row["event_id"].strip()
        pid = row["process_id"].strip()
        etype = row["event_type"].strip().upper()
        other = row["message_to"].strip()     # SEND: receiver | RECEIVE: sender

        if pid not in clocks:                 # first time we see this process
            clocks[pid] = int(row["initial_clock"])
        old = clocks[pid]

        if etype == "SEND":
            clocks[pid] = old + 1
            queues.setdefault((pid, other), []).append(clocks[pid])
            print(f"{eid}: {pid} SEND to {other}: L = {old} + 1 = {clocks[pid]}"
                  f"  (message carries {clocks[pid]})")
        elif etype == "RECEIVE":
            waiting = queues.get((other, pid), [])
            t_msg = waiting.pop(0) if waiting else 0
            clocks[pid] = max(old, t_msg) + 1
            print(f"{eid}: {pid} RECEIVE from {other}: "
                  f"L = max({old}, {t_msg}) + 1 = {clocks[pid]}")
        else:
            clocks[pid] = old + 1
            print(f"{eid}: {pid} INTERNAL: L = {old} + 1 = {clocks[pid]}")

        log.append((clocks[pid], pid, eid))
```

```
print("\nFinal clock of each process:")
for pid in sorted(clocks):
    print(" ", pid, "=", clocks[pid])

print("\nFinal causal (total) order - sorted by timestamp, ties by process id:")
for ts, pid, eid in sorted(log):
    print(f"  {eid}  {pid}  Timestamp: {ts}")
```

Sample output

```
Step-by-step clock updates
E1: P1 SEND to P2: L = 0 + 1 = 1  (message carries 1)
E2: P2 INTERNAL: L = 0 + 1 = 1
E3: P2 RECEIVE from P1: L = max(1, 1) + 1 = 2
E4: P2 SEND to P3: L = 2 + 1 = 3  (message carries 3)
E5: P3 RECEIVE from P2: L = max(0, 3) + 1 = 4
E6: P1 INTERNAL: L = 1 + 1 = 2
E7: P3 SEND to P1: L = 4 + 1 = 5  (message carries 5)
E8: P1 RECEIVE from P3: L = max(2, 5) + 1 = 6

Final clock of each process:
  P1 = 6
  P2 = 3
  P3 = 5

Final causal (total) order - sorted by timestamp, ties by process id:
  E1  P1  Timestamp: 1
  E2  P2  Timestamp: 1
  E6  P1  Timestamp: 2
  E3  P2  Timestamp: 2
  E4  P2  Timestamp: 3
  E5  P3  Timestamp: 4
  E7  P3  Timestamp: 5
  E8  P1  Timestamp: 6
```

If the question changes

If the question says...	Change this
RECEIVE rows name the sender in message_to	That is what this program expects (see the sample CSV).
Your chit lists a RECEIVE row with its own convention	Change only the key used in `queues.get((other, pid), [])`.
Event type written LOCAL / INTERNAL / any other word	Already handled. Anything that is not SEND or RECEIVE is internal.
Initial clocks not zero	Already handled. Each process takes its own initial_clock.

Watch out

- Put the sender's ID in `message_to` for RECEIVE rows and a dash for internal events. Write this in your CSV.
- A SEND must appear before its RECEIVE in the file, or the message is counted as 0.

7. Vector Clocks

Program file	`vector_clock.py` (32 chits of this type)
CSV file	`vector_events_6.csv` (the number changes in each chit)
Columns	`event_id, process_id, event_action, target_process`
Chit wording	Track vector clocks [V1, V2, V3] from CSV and decide whether pairs of events are causally related or concurrent.
Output must show	Vector after every event, then Causally Preceding (V1 < V2) or Concurrent (E1 || E2)

Formulas / rules

- Every event: own component + 1
- SEND: attach the vector to the message
- RECEIVE: element-wise max(own, received), then own component + 1
- A < B if every A[i] <= B[i] and A != B. Otherwise, if neither is smaller: concurrent

How it works

- The process list is built from the CSV, sorted. Each process gets one vector slot. Any number of processes works.
- SEND stores a copy of the sender's vector. RECEIVE takes the oldest waiting copy from that sender.
- Every pair of events is compared. If one vector is <= the other everywhere, one event happened before the other. Otherwise they are concurrent.

Sample CSV (vector_events_6.csv)

```
event_id,process_id,event_action,target_process
E1,P1,SEND,P2
E2,P2,RECEIVE,P1
E3,P3,INTERNAL,-
E4,P2,SEND,P3
E5,P3,RECEIVE,P2
E6,P1,INTERNAL,-
```

Program (vector_clock.py)

```
import csv
from itertools import combinations

CSV_FILE = "vector_events_6.csv"      # <-- change the filename here

with open(CSV_FILE, "r") as f:
    rows = [{k: v.strip() for k, v in r.items()}
            for r in csv.DictReader(f, skipinitialspace=True)]

processes = sorted({r["process_id"] for r in rows})   # any number of processes
n = len(processes)
clocks = {p: [0] * n for p in processes}
queues = {}          # (sender, receiver) -> vectors in transit
events = {}          # event id -> vector after the event

print("Processes:", processes)
for r in rows:
    eid = r["event_id"]
    pid = r["process_id"]
    action = r["event_action"].upper()
    other = r["target_process"]        # SEND: receiver | RECEIVE: sender
    i = processes.index(pid)

    if action == "SEND":
        clocks[pid][i] += 1
        queues.setdefault((pid, other), []).append(clocks[pid].copy())
        what = f"SEND to {other}"
    elif action == "RECEIVE":
        waiting = queues.get((other, pid), [])
        got = waiting.pop(0) if waiting else [0] * n
        clocks[pid] = [max(a, b) for a, b in zip(clocks[pid], got)]
        clocks[pid][i] += 1
        what = f"RECEIVE from {other} (got {got})"
    else:
        clocks[pid][i] += 1
```

```
        what = "INTERNAL"

    events[eid] = clocks[pid].copy()
    print(f"{eid}: {pid} {what} -> {events[eid]}")

def leq(a, b):
    return all(x <= y for x, y in zip(a, b))

print("\nCausality check for every pair of events:")
for e1, e2 in combinations(events, 2):
    v1, v2 = events[e1], events[e2]
    if v1 == v2:
        print(f"{e1} = {e2}  Equal {v1}")
    elif leq(v1, v2):
        print(f"{e1} -> {e2}  Causally Preceding ({v1} < {v2})")
    elif leq(v2, v1):
        print(f"{e2} -> {e1}  Causally Preceding ({v2} < {v1})")
    else:
        print(f"{e1} || {e2}  Concurrent ({v1} vs {v2})")
```

Sample output

```
Processes: ['P1', 'P2', 'P3']
E1: P1 SEND to P2 -> [1, 0, 0]
E2: P2 RECEIVE from P1 (got [1, 0, 0]) -> [1, 1, 0]
E3: P3 INTERNAL -> [0, 0, 1]
E4: P2 SEND to P3 -> [1, 2, 0]
E5: P3 RECEIVE from P2 (got [1, 2, 0]) -> [1, 2, 2]
E6: P1 INTERNAL -> [2, 0, 0]

Causality check for every pair of events:
E1 -> E2  Causally Preceding ([1, 0, 0] < [1, 1, 0])
E1 || E3  Concurrent ([1, 0, 0] vs [0, 0, 1])
E1 -> E4  Causally Preceding ([1, 0, 0] < [1, 2, 0])
E1 -> E5  Causally Preceding ([1, 0, 0] < [1, 2, 2])
E1 -> E6  Causally Preceding ([1, 0, 0] < [2, 0, 0])
E2 || E3  Concurrent ([1, 1, 0] vs [0, 0, 1])
E2 -> E4  Causally Preceding ([1, 1, 0] < [1, 2, 0])
E2 -> E5  Causally Preceding ([1, 1, 0] < [1, 2, 2])
E2 || E6  Concurrent ([1, 1, 0] vs [2, 0, 0])
E3 || E4  Concurrent ([0, 0, 1] vs [1, 2, 0])
E3 -> E5  Causally Preceding ([0, 0, 1] < [1, 2, 2])
E3 || E6  Concurrent ([0, 0, 1] vs [2, 0, 0])
E4 -> E5  Causally Preceding ([1, 2, 0] < [1, 2, 2])
E4 || E6  Concurrent ([1, 2, 0] vs [2, 0, 0])
E5 || E6  Concurrent ([1, 2, 2] vs [2, 0, 0])
```

If the question changes

If the question says...	Change this
Only specific events must be compared (for example E2 and E5)	After the loop add `print(leq(events["E2"], events["E5"]))` or just print that pair using the same if/elif chain.
More than 3 processes	No change.
Show the final vector of each process	After the loop `for p in processes: print(p, clocks[p])`.
Different process names	No change. Names come from the CSV.

Watch out

- For RECEIVE rows, `target_process` is the sender. Say so in your CSV.
- Print vectors as lists [v1, v2, v3]. The program does.

8. Multithreading with a Mutex (Lock)

Program file	`multithread.py` (34 chits of this type)
CSV file	`tasks_4.csv` (the number changes in each chit)
Columns	`task_id, thread_name, burst_ms, priority, status`
Chit wording	Multithreaded worker simulation using thread locks (mutex) to process tasks without race conditions.
Output must show	Thread execution logs, lock acquired / released, final completed task list

How it works

- `threading.Thread(target=..., args=...)` creates a thread. `start()` runs it. `join()` waits for it.
- The shared data (the `completed` list and the `counter`) are changed only inside `with lock:`. That is the critical section.
- The task's work (`time.sleep(burst)`) is outside the lock so threads really overlap.
- The output order differs on each run. That is normal for threads.

Sample CSV (tasks_4.csv)

```
task_id,thread_name,burst_ms,priority,status
T1,Worker_1,200,HIGH,PENDING
T2,Worker_2,150,MEDIUM,PENDING
T3,Worker_3,100,LOW,PENDING
T4,Worker_1,120,HIGH,PENDING
T5,Worker_2,80,LOW,PENDING
```

Program (multithread.py)

```
import csv
import threading
import time

CSV_FILE = "tasks_4.csv"              # <-- change the filename here
SORT_BY_PRIORITY = False              # True -> HIGH tasks are started first

tasks = []
with open(CSV_FILE, "r") as f:
    for row in csv.DictReader(f, skipinitialspace=True):
        tasks.append({
            "id": row["task_id"].strip(),
            "thread": row["thread_name"].strip(),
            "burst": int(row["burst_ms"]),
            "priority": row["priority"].strip().upper(),
            "status": row["status"].strip().upper(),
        })

if SORT_BY_PRIORITY:
    rank = {"HIGH": 0, "MEDIUM": 1, "LOW": 2}
    tasks.sort(key=lambda t: rank.get(t["priority"], 9))

lock = threading.Lock()     # the MUTEX
completed = []              # shared resource 1
counter = 0                 # shared resource 2

def worker(task):
    global counter
    name = task["thread"]
    print(f"[{name}] started task {task['id']} (burst {task['burst']} ms)")
    time.sleep(task["burst"] / 1000)          # the work itself, outside the lock

    print(f"[{name}] waiting for lock ({task['id']})")
    with lock:                                # acquire ... release automatically
        print(f"[{name}] ACQUIRED lock ({task['id']})")
        counter += 1                          # critical section: shared data
        completed.append(task["id"])
        task["status"] = "COMPLETED"
        print(f"[{name}] finished {task['id']}, counter = {counter}")
        print(f"[{name}] RELEASING lock")

threads = []
for t in tasks:
    th = threading.Thread(target=worker, args=(t,), name=t["thread"])
    threads.append(th)
```

```
    th.start()

for th in threads:
    th.join()                                # wait for all threads

print("\nAll threads finished.")
print("Completion order:", completed)
print("Total completed:", counter)
for t in tasks:
    print(f"  {t['id']}  {t['thread']}  {t['priority']}  {t['status']}")
```

Optional: one thread per thread_name (replace the thread start loop)

```
groups = {}
for t in tasks:
    groups.setdefault(t["thread"], []).append(t)

def run_group(mine):
    for task in mine:
        worker(task)          # same worker() as above

threads = [threading.Thread(target=run_group, args=(g,), name=n)
           for n, g in groups.items()]
for th in threads:
    th.start()
```

Sample output

```
[Worker_1] started task T1 (burst 200 ms)
[Worker_2] started task T2 (burst 150 ms)
[Worker_3] started task T3 (burst 100 ms)
[Worker_1] started task T4 (burst 120 ms)
[Worker_2] started task T5 (burst 80 ms)
[Worker_2] waiting for lock (T5)
[Worker_2] ACQUIRED lock (T5)
[Worker_2] finished T5, counter = 1
[Worker_2] RELEASING lock
[Worker_3] waiting for lock (T3)
[Worker_3] ACQUIRED lock (T3)
[Worker_3] finished T3, counter = 2
[Worker_3] RELEASING lock
[Worker_1] waiting for lock (T4)
[Worker_1] ACQUIRED lock (T4)
[Worker_1] finished T4, counter = 3
[Worker_1] RELEASING lock
[Worker_2] waiting for lock (T2)
[Worker_2] ACQUIRED lock (T2)
[Worker_2] finished T2, counter = 4
[Worker_2] RELEASING lock
[Worker_1] waiting for lock (T1)
[Worker_1] ACQUIRED lock (T1)
[Worker_1] finished T1, counter = 5
[Worker_1] RELEASING lock

All threads finished.
Completion order: ['T5', 'T3', 'T4', 'T2', 'T1']
Total completed: 5
  T1  Worker_1  HIGH  COMPLETED
  T2  Worker_2  MEDIUM  COMPLETED
  T3  Worker_3  LOW  COMPLETED
  T4  Worker_1  HIGH  COMPLETED
  T5  Worker_2  LOW  COMPLETED
```

If the question changes

If the question says...	Change this
Priority scheduling	Set `SORT_BY_PRIORITY = True`.
Show the race condition without the lock	Replace the update by `tmp = counter; time.sleep(0.001); counter = tmp + 1` and remove `with lock:`. Counter ends below the real total.
One thread per thread_name (not per task)	See the grouping snippet below.
Shared balance / stock instead of a counter	Change the line inside the lock that updates the shared value.

Watch out

- Never move the shared updates outside the lock.
- Print the lock messages: waiting, ACQUIRED, RELEASING. The chit asks for them.

9. Load Balancing - Round Robin and Least Connections

Program file	`load_balancer.py` (33 chits of this type)
CSV file	`workload_5.csv` (the number changes in each chit)
Columns	`job_id, compute_units, assigned_worker, status`
Chit wording	Dynamic load balancing simulator (Round-Robin or Least-Connections) that distributes jobs from CSV among worker nodes.
Output must show	Job allocations, current load per worker, balanced summary

How it works

- Set `ALGORITHM` at the top: **ROUND_ROBIN**, **LEAST_CONNECTIONS** or **LEAST_LOAD**.
- Round Robin: job number `i` goes to worker `i % number_of_workers`.
- Least Connections: the worker with the fewest jobs so far. Least Load: the worker with the smallest total compute units.
- After each assignment the program prints the current load of every worker, then a final summary.

Sample CSV (workload_5.csv)

```
job_id,compute_units,assigned_worker,status
J101,35,UNASSIGNED,PENDING
J102,60,UNASSIGNED,PENDING
J103,20,UNASSIGNED,PENDING
J104,45,UNASSIGNED,PENDING
J105,10,UNASSIGNED,PENDING
J106,80,UNASSIGNED,PENDING
J107,25,UNASSIGNED,PENDING
```

Program (load_balancer.py)

```
import csv

CSV_FILE = "workload_5.csv"                       # <-- change the filename here
ALGORITHM = "ROUND_ROBIN"      # ROUND_ROBIN | LEAST_CONNECTIONS | LEAST_LOAD
WORKERS = ["Worker_1", "Worker_2", "Worker_3"]    # add / remove workers here

jobs = []
with open(CSV_FILE, "r") as f:
    for row in csv.DictReader(f, skipinitialspace=True):
        jobs.append({
            "id": row["job_id"].strip(),
            "units": int(row["compute_units"]),
            "worker": row["assigned_worker"].strip(),
            "status": row["status"].strip(),
        })

job_count = {w: 0 for w in WORKERS}      # connections = jobs assigned
total_load = {w: 0 for w in WORKERS}     # sum of compute units
assigned = {w: [] for w in WORKERS}

print("Algorithm:", ALGORITHM)
print("Incoming job allocations")
for i, job in enumerate(jobs):
    if ALGORITHM == "ROUND_ROBIN":
        w = WORKERS[i % len(WORKERS)]
    elif ALGORITHM == "LEAST_CONNECTIONS":
        w = min(WORKERS, key=lambda x: job_count[x])
    else:                                                # LEAST_LOAD
        w = min(WORKERS, key=lambda x: total_load[x])

    job["worker"] = w
    job["status"] = "ASSIGNED"
    job_count[w] += 1
    total_load[w] += job["units"]
    assigned[w].append(job["id"])

    loads = "  ".join(f"{x}={total_load[x]}" for x in WORKERS)
    print(f"{job['id']} ({job['units']} units) -> {w}   | load: {loads}")

print("\nBalanced summary")
for w in WORKERS:
    print(f"{w}: jobs={job_count[w]}  total load={total_load[w]}  {assigned[w]}")
```

```
print("Load difference (max - min):", max(total_load.values()) - min(total_load.values()))

print("\nFinal job table")
for job in jobs:
    print(job)
```

Sample output

```
Algorithm: ROUND_ROBIN
Incoming job allocations
J101 (35 units) -> Worker_1   | load: Worker_1=35  Worker_2=0  Worker_3=0
J102 (60 units) -> Worker_2   | load: Worker_1=35  Worker_2=60  Worker_3=0
J103 (20 units) -> Worker_3   | load: Worker_1=35  Worker_2=60  Worker_3=20
J104 (45 units) -> Worker_1   | load: Worker_1=80  Worker_2=60  Worker_3=20
J105 (10 units) -> Worker_2   | load: Worker_1=80  Worker_2=70  Worker_3=20
J106 (80 units) -> Worker_3   | load: Worker_1=80  Worker_2=70  Worker_3=100
J107 (25 units) -> Worker_1   | load: Worker_1=105  Worker_2=70  Worker_3=100

Balanced summary
Worker_1: jobs=3  total load=105  ['J101', 'J104', 'J107']
Worker_2: jobs=2  total load=70  ['J102', 'J105']
Worker_3: jobs=2  total load=100  ['J103', 'J106']
Load difference (max - min): 35

Final job table
{'id': 'J101', 'units': 35, 'worker': 'Worker_1', 'status': 'ASSIGNED'}
{'id': 'J102', 'units': 60, 'worker': 'Worker_2', 'status': 'ASSIGNED'}
{'id': 'J103', 'units': 20, 'worker': 'Worker_3', 'status': 'ASSIGNED'}
{'id': 'J104', 'units': 45, 'worker': 'Worker_1', 'status': 'ASSIGNED'}
{'id': 'J105', 'units': 10, 'worker': 'Worker_2', 'status': 'ASSIGNED'}
{'id': 'J106', 'units': 80, 'worker': 'Worker_3', 'status': 'ASSIGNED'}
{'id': 'J107', 'units': 25, 'worker': 'Worker_1', 'status': 'ASSIGNED'}
```

If the question changes

If the question says...	Change this
Different number or names of workers	Edit the `WORKERS` list.
Chit says Round-Robin / says Least-Connections	Change `ALGORITHM` only.
Load means compute units	Use `LEAST_LOAD`.
Jobs finish while others arrive	Subtract from `job_count[w]` when a job completes, then Least Connections differs from Round Robin.

Watch out

- Without completions, Least Connections gives the same result as Round Robin (all jobs stay assigned). Say this if asked.
- Least Connections counts **jobs**. Least Load adds up **compute units**. They are different.

10. MPI Matrix Multiplication - Scatter and Gather

Program file	`mpi_matrix.py` (33 chits of this type)
CSV file	`matrix_22.csv` (the number changes in each chit)
Columns	`row_id, col_0, col_1, col_2, col_3`
Chit wording	Parallel matrix multiplication using a simulated MPI Scatter-Gather pattern with multiprocessing/threads; compute row dot products.
Output must show	Master scattering rows -> workers computing dot products -> master gathering the product

How it works

- **Master** reads the matrix from the CSV (the `row_id` column is ignored).
- **Scatter**: worker `i` is given row `i`. Each worker computes one row of the result, one dot product per column.
- **Gather**: `pool.map` returns the rows in order, and the master prints the final matrix.
- The CSV holds one matrix, so the program calculates **A x A**. The matrix must be square.

Sample CSV (matrix_22.csv)

```
row_id,col_0,col_1,col_2,col_3
R0,2,4,1,5
R1,1,0,3,2
R2,4,1,2,0
R3,3,2,1,1
```

Program (mpi_matrix.py)

```
import csv
from concurrent.futures import ThreadPoolExecutor

CSV_FILE = "matrix_22.csv"            # <-- change the filename here

matrix = []
with open(CSV_FILE, "r") as f:
    for row in csv.DictReader(f, skipinitialspace=True):
        matrix.append([int(row[c]) for c in row if c.strip() != "row_id"])

n = len(matrix)
if any(len(r) != n for r in matrix):
    print("Matrix must be square to compute A x A")
    raise SystemExit

B = matrix          # second matrix. Change if the question gives a separate B

print("Matrix A (read by MASTER):")
for r in matrix:
    print("  ", r)

def worker(i):
    row = matrix[i]                           # row i was "scattered" to this worker
    result = []
    lines = [f"Worker {i} received row {i}: {row}"]
    for j in range(n):
        terms = [f"{row[k]}*{B[k][j]}" for k in range(n)]
        value = sum(row[k] * B[k][j] for k in range(n))
        lines.append(f"  C[{i}][{j}] = " + " + ".join(terms) + f" = {value}")
        result.append(value)
    print("\n".join(lines) + "\n", end="")           # one single write keeps the log tidy
    return result

print("\nMASTER: scattering rows to workers")
with ThreadPoolExecutor(max_workers=n) as pool:
    result = list(pool.map(worker, range(n)))    # map keeps the row order

print("\nMASTER: gathering results from workers")
print("Final matrix product C = A x A:")
for r in result:
    print("  ", r)
```

Sample output

```
Matrix A (read by MASTER):
  [2, 4, 1, 5]
```

```
  [1, 0, 3, 2]
  [4, 1, 2, 0]
  [3, 2, 1, 1]

MASTER: scattering rows to workers
Worker 0 received row 0: [2, 4, 1, 5]
  C[0][0] = 2*2 + 4*1 + 1*4 + 5*3 = 27
  C[0][1] = 2*4 + 4*0 + 1*1 + 5*2 = 19
  C[0][2] = 2*1 + 4*3 + 1*2 + 5*1 = 21
  C[0][3] = 2*5 + 4*2 + 1*0 + 5*1 = 23
Worker 1 received row 1: [1, 0, 3, 2]
  C[1][0] = 1*2 + 0*1 + 3*4 + 2*3 = 20
  C[1][1] = 1*4 + 0*0 + 3*1 + 2*2 = 11
  C[1][2] = 1*1 + 0*3 + 3*2 + 2*1 = 9
  C[1][3] = 1*5 + 0*2 + 3*0 + 2*1 = 7
Worker 3 received row 3: [3, 2, 1, 1]
  C[3][0] = 3*2 + 2*1 + 1*4 + 1*3 = 15
  C[3][1] = 3*4 + 2*0 + 1*1 + 1*2 = 15
  C[3][2] = 3*1 + 2*3 + 1*2 + 1*1 = 12
  C[3][3] = 3*5 + 2*2 + 1*0 + 1*1 = 20
Worker 2 received row 2: [4, 1, 2, 0]
  C[2][0] = 4*2 + 1*1 + 2*4 + 0*3 = 17
  C[2][1] = 4*4 + 1*0 + 2*1 + 0*2 = 18
  C[2][2] = 4*1 + 1*3 + 2*2 + 0*1 = 11
  C[2][3] = 4*5 + 1*2 + 2*0 + 0*1 = 22

MASTER: gathering results from workers
Final matrix product C = A x A:
  [27, 19, 21, 23]
  [20, 11, 9, 7]
  [17, 18, 11, 22]
  [15, 15, 12, 20]
```

If the question changes

If the question says...	Change this
The question gives a second matrix B	Read B from a second CSV in the same way and set `B = matrix_b`. Workers already use `B[k][j]`.
Different number of workers	Change `max_workers`.
Real processes instead of threads	Use `ProcessPoolExecutor`. Put the main code under `if __name__ == "__main__":` (needed on Windows).
Real MPI (mpi4py)	Needs mpi4py and an MPI install. The chit says simulated, so the thread version is fine.

Watch out

- Order of the worker messages can change between runs. The final matrix never does.
- The sample result for row 0 is [27, 19, 21, 23]. Check by hand once.

Last-minute revision

Five things to identify in every new chit

1. CSV file name.
2. CSV column names.
3. Type conversion of each column (int, float, TRUE/FALSE, text).
4. The main algorithm rule.
5. The required output labels and any extra condition.

Do not change blindly

- The algorithm's formula or rule (for example `max(L, T_msg) + 1`, or RTT / 2).
- The order of send and receive operations.
- The lock around shared data in the multithreading program.
- The message rules: ELECTION goes to **higher** IDs in Bully, around the ring in Ring.

One-line summary of each algorithm

Program	Core idea to say out loud
1 RPC	Marshal the arguments, look the method up in a dispatcher table, run it on the server, return the result.
2 Bully	Send ELECTION to higher IDs. Any alive higher node answers OK and takes over. No OK means you win and announce COORDINATOR.
3 Ring	Pass an ELECTION token round the ring, skipping dead nodes. When it returns, the highest ID wins. Then circulate COORDINATOR.
4 Cristian	RTT = T1 - T0. Server time now = T_server + RTT / 2. Offset = that value - T1.
5 Berkeley	Poll all clocks, drop the faulty ones, average the rest, send each node Average - its clock.
6 Lamport	Send and internal: L + 1. Receive: max(L, T_msg) + 1. Sort by timestamp.
7 Vector	Own slot + 1 on every event. Receive: element-wise max first. Compare: all <= means before, otherwise concurrent.
8 Multithreading	One thread per task. Lock around the shared list and counter. start(), then join().
9 Load balancing	Round Robin: i mod workers. Least Connections: fewest jobs. Least Load: fewest compute units.
10 MPI	Scatter one row to each worker, each computes a row of the product, gather the rows in order.

Practice plan

- Day 1: set up the folder in VS Code. Run all 10 programs with their sample CSVs.
- Day 2: type each program from a blank file without looking. Fix what you forgot.
- Day 3: pick random chits from the question bank. For each one write the CSV, change the file name, run it, and compare with the chit's expected output.
- Most useful drill: for each program, change one requirement (different initiator, priority instead of ID, a second matrix, seconds instead of ms) and make it work.